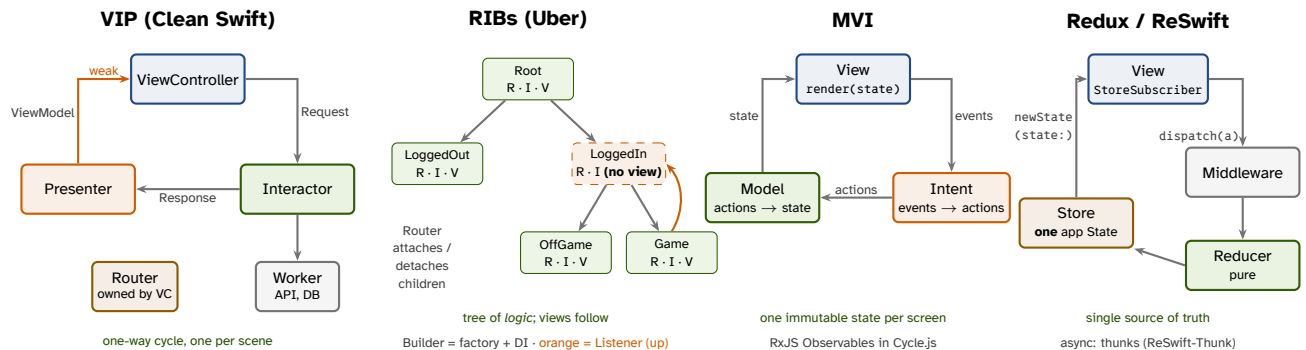


In one line: Every iOS architecture answers three questions: **who owns the state**, **which way does data flow**, and **where do side effects live**. **VIP** (Clean Swift) makes each screen a one-way cycle of three objects; **RIBs** (Uber) drive the app by a *tree of business logic*, not of views; **MVI** and **Redux** keep one immutable state changed only by a reducer; **TCA** is Redux made composable; **MV** says SwiftUI views plus `@observable` models are already enough.



Pattern	State lives in	Data flow	Testability	Boilerplate	Fits
VIP	Interactor (+ DataStore)	VC → I → P → VC, one way	high: spy the next role	high: 7 per scene	UIKit apps, per-scene rigour
RIBs	each RIB's Interactor	tree; Rx down, listener up	high: I/R/B behind protocols	very high, codegen	huge apps, hundreds of engineers
MVI	one immutable screen State	Intent → Model → View	high: pure reducer	medium	complex screens, Android parity
Redux / ReSwift	one global Store	dispatch → reducer → subscribers	high reducers; effects harder	medium-high	shared app-wide state
TCA	Store tree of features	unidirectional, composed	very high: TestStore	high	big SwiftUI teams
MV	<code>@observable</code> models	view calls model, observes it	models yes, view logic no	low	SwiftUI small-mid apps



How it works

- VIP** (Clean Swift; Clean Architecture per scene): VC (DisplayLogic) sends a *Request* to the Interactor (BusinessLogic, uses **Workers**, holds the DataStore); its *Response* goes to the Presenter (PresentationLogic), which formats a *ViewModel* and calls `display...` on the VC. **Router** (RoutingLogic + DataPassing) navigates, passing data via the DataStore. Models nest per use case (ListOrders.Fetch.Request). Older templates: ...Input/...Output protocols, wired by a **Configurator**.
- RIBs** = Router · Interactor · Builder (+ Component for dependencies, optional Presenter and View). The Interactor holds business logic and Rx subscriptions, alive between `didBecomeActive()` and `willResignActive()`; bind with `disposeOnDeactivate(interactor:)`. Router attaches/detaches children; Builder creates the unit and is the only part that knows the DI system. Up: a Listener the parent implements; down: Rx streams or `build()` arguments.
- MVI**: *Intent* turns user events into actions, *Model* turns actions into state, *View* renders state — a pure `render(state)` of one immutable state.
- Redux**'s 3 principles: one store = single source of truth · state is read-only, changed only by an action · pure reducers (state + action → new state). ReSwift:
`Store<AppState>(reducer: state:), dispatch, newState(state:); subscribe(self) { $0.select(...) }`
narrows updates, skipping repeats of an Equatable substate.
- MV**: no per-screen *ViewModel*; logic lives in `@observable` models (iOS 17, macOS 14). A view updates only when a property its body reads changes. Own with `@State`, share with `.environment(model)`, read with `@Environment(Model.self)`, bind with `@Bindable`.
- TCA**: State · Action · Reducer · Store; effects and dependencies are injectable, and `TestStore` asserts every state change and effect.

Why RIBs exist

Uber's rider app (rewrite, 2016): an MVC app grown to a few hundred engineers, one `RequestViewController` from 300 to 3,000+ lines. RIBs make **business logic**, **not the view tree**, **drive routing** — a deep logic tree with a shallow view tree.

Example — one VIP use case

```
enum ListOrders { enum Fetch { // per use case
    struct Request {}
    struct Response { let orders: [Order] }
    struct ViewModel { let rows: [String] } }
protocol ListOrdersDisplayLogic: AnyObject {
    func display(_ vm: ListOrders.Fetch.ViewModel) }
final class ListOrdersInteractor { // BusinessLogic
    var presenter: ListOrdersPresenter?; let worker = OrdersWorker()
    func fetch(_ r: ListOrders.Fetch.Request) {
        worker.load { self.presenter?.present(.init(orders: $0)) } }
final class ListOrdersPresenter { // PresentationLogic
    weak var viewController: ListOrdersDisplayLogic? // no cycle
    func present(_ r: ListOrders.Fetch.Response) {
        viewController?.display(.init(rows: r.orders.map(\.title))) } }
```

Traps

- ! "VIP is VIPER renamed." VIPER (2014): Presenter *between* View and Interactor, both ways; Entities stay in the Interactor. VIP: a one-way cycle.
- ! RIBs' Router is not "just navigation": it attaches/detaches child RIBs from Interactor output — screens follow the tree.
- ! Redux: subscribers hear every change unless they select a slice; local UI state (`isDropdownOpen`) stays out of the store.
- ! A one-off message kept in state shows again on every re-render until the view reports it shown and the state clears it.
- ! Conforming to the Observable protocol alone adds no observation — use the `@Observable` macro.